

2024 Information Retrieval and Extraction HW3

312553024 江尚軒

Questions

1. What kind of pre-processing did you apply to the photo or dialogue text? Additionally, please discuss how different preprocessing methods affected the performance of the models?

Dialogue to text

Concatenate the messages in the dialogue into a single string, separated by spaces.

```
def convert_dialogue_to_text(dialogue):  
    """Convert dialogue messages into a single text string"""  
    messages = [msg["message"] for msg in dialogue if msg["message"]]  
    return " ".join(messages)
```

Description to text

Remove "Objects in the photo: " from the photo description.

```
def convert_description_to_text(description):  
    """Convert description into a single text string"""  
    if "Objects in the photo: " in description:  
        description = description.replace("Objects in the photo: ", "")  
    return description
```

Preprocess text

Preprocess text with the following steps:

1. Convert to lowercase
2. Remove special characters
3. Tokenize and remove stopwords
4. Lemmatize tokens

```
def preprocess_text(text):
    """Preprocess text with following steps:
    1. Convert to lowercase
    2. Remove special characters
    3. Tokenize and remove stopwords
    4. Lemmatize tokens
    """
    # Convert to lowercase
    text = text.lower()

    # Remove special characters
    text = re.sub(r'^\w\s', ' ', text)

    # Tokenize
    tokens = word_tokenize(text)

    # Remove stopwords
    stop_words = set(stopwords.words('english'))
    tokens = [token for token in tokens if token not in stop_words]

    # Lemmatize
    lemmatizer = WordNetLemmatizer()
    tokens = [lemmatizer.lemmatize(token) for token in tokens]

    return ' '.join(tokens)
```

Please discuss how different preprocessing methods affected the performance of the models?

The experimental results show that the preprocessing improves performance because it removes unnecessary noise, allowing TF-IDF to more accurately retrieve the relevance between dialogue and description.

✓	TF-IDF_2.csv Complete · 2d ago · TF-IDF with preprocessing	0.67666	☐
✓	TF-IDF_1.csv Complete · 2d ago · TF-IDF without preprocessing	0.64666	☐

2. How did you align the photo and dialogue text in the same embedding space? Use pre-trained model or train your own?

How did you align the photo and dialogue text in the same embedding space?

Use an image-to-text model to generate captions for the images. Then, utilize TF-IDF to retrieve the relevance between the dialogue and these image captions.

```
# Open image
image = Image.open(image_path)

# Process image
inputs = processor(images=image, return_tensors="pt")
# Move inputs to same device as model
inputs = {k: v.to(device) for k, v in inputs.items()}
# Generate caption
outputs = model.generate(
    **inputs, # Pass all inputs directly
    max_length=50,
    num_return_sequences=1
)

# Move outputs back to CPU for decoding
outputs = outputs.cpu()
caption = processor.decode(outputs[0], skip_special_tokens=True)

# Update photo description
data['photo_description'] = caption
```

Use pre-trained model or train your own?

Use pre-trained models from Hugging Face, such as [git-base-coco](#), [git-large-coco](#), [blip-image-captioning-base](#), and [blip-image-captioning-large](#).



TF-IDF_3.csv

Complete · 1d ago · TF-IDF with git-base_1

0.30666



3. Please discuss based on your experimental results. How do you improve the performance of your model? (e.g. add a module or try different models and observing performance changes). What was the result?

How do you improve the performance of your model?

The experimental results show that using only image captions performs worse than using the original photo descriptions. Therefore, I concatenate the photo descriptions and image captions together and then use TF-IDF to calculate their relevance to the dialogue.

```
# Get caption from second file if available
capt_desc = captioned_data.get(photo_id, {}).get('photo_description', '')

# Create combined item preserving all metadata
combined_item = orig_item.copy()
combined_item['photo_description'] = f"{orig_item['photo_description']}. Caption: {capt_desc}".strip()

# Write to output file
json.dump(combined_item, f)
f.write('\n')
```

What was the result?

The experiments reveal that concatenating the photo descriptions and image captions significantly improves Recall@30 performance. Moreover, the more image captions concatenated, the better the performance. However, when the number of concatenated image captions exceeds 3, performance begins to decline, indicating that too many concatenations introduce excessive noise.

	TF-IDF_17.csv Complete · 4m ago · TF-IDF with base+blip-image-captioning-large_1+git-large-coco_1+git-bas...	0.74666	☒
	TF-IDF_16.csv Complete · 11m ago · TF-IDF with base+blip-image-captioning-large_1+git-large-coco_1+git-bas...	0.75000	☒
	TF-IDF_15.csv Complete · 1h ago · TF-IDF with base+blip-image-captioning-large_1+git-base-coco_1	0.75333	☒
	TF-IDF_14.csv Complete · 1h ago · TF-IDF with base+blip-image-captioning-large_1+git-large-coco_1	0.73666	☐
	TF-IDF_13.csv Complete · 18h ago · TF-IDF with base+blip-image-captioning-large_1	0.74666	☐
	TF-IDF_11.csv Complete · 18h ago · TF-IDF with base+git-large-coco_1	0.73333	☐
	TF-IDF_8.csv Complete · 18h ago · TF-IDF with base+git-base-coco_1	0.74000	☐
	TF-IDF_3.csv Complete · 1d ago · TF-IDF with git-base_1	0.30666	☐